Personal Finance Advisor Bot

Reinforcement Learning & AI-Driven Personal Finance Advisory & Management Platform

Project Documentation

AI-Powered Web Platform for 50/30/20 Budgeting, Expense Tracking, Goal-Based Savings and Real-Time Financial Advice

Project Description

Personal Finance Advisor Bot is an AI-driven web-based personal finance advisory and management platform designed to help users understand where their money goes and make better day-to-day financial decisions. The platform brings several tools into one interface: an expense and inflow tracker, a 50/30/20 budget planner, a goal-based savings hub, an AI advisor chatbot, a monthly statement view and a "Can I Afford This?" purchase checker. It gives users a simple and interactive way to log their finances and receive useful, instant insights.

The dashboard calculates monthly inflow, total expenses, net cash flow and savings rate, and automatically flags overspending in categories such as Shopping, Food & Dining and Subscriptions. The 50/30/20 module divides spending into Needs, Wants and Savings, tracks spending against a monthly limit for each category, and shows a colour-coded progress bar. The Savings Goals module tracks targets such as an emergency buffer or a laptop fund and calculates the monthly pace required to reach each goal on time.

The system supports multiple user profiles (Alex, Maya and Marcus), each with a different financial situation, and a selectable currency (USD shown). Alex is a full-time salaried professional with a bi-weekly paycheck who wants to curb lifestyle creep, while Maya is a student managing campus expenses, tuition-related costs and savings goals. The application is built with a Flask (Python) backend and uses Gemini AI for real-time financial advice. The platform is also designed around a Reinforcement Learning (RL) perspective, in which the advisor treats a user's budget status as a state, a recommendation as an action, and improved savings behaviour as the reward, so that its advice can be refined over time. Overall, the project demonstrates how web development, financial calculations, AI assistance and adaptive decision-making can be combined into one user-friendly financial management platform.

Project Scenarios

Scenario 1: Expense and Inflow Tracking

A user logs an expense with a description, category, payment method (Card or Digital Wallet) and amount, or records an income inflow such as salary. The entry is added to a dated transaction list, and any note, for example a category slightly over its limit, is displayed under the entry. Entries can be deleted if recorded by mistake.

Scenario 2: 50/30/20 Budget Monitoring

The user opens the budget view and sees each category (Housing, Food & Dining, Groceries, Transportation, Utilities, Entertainment, Healthcare, Shopping, Education) tagged as a Need or a Want, with spending to date, percentage of the limit used, and an editable monthly limit.

Scenario 3: Overspending Detection

When spending in a category exceeds its planned limit, the dashboard shows an alert listing the categories and the amount by which each is exceeded, along with actions to get AI cut-back advice or adjust the budget limits.

Scenario 4: Goal-Based Savings

The user creates a goal with a name, type, target amount and target date. The hub shows the amount saved, percentage achieved and the monthly pace needed, and summarises overall progress across all active goals.

Scenario 5: AI Financial Guidance

The user asks the AI Advisor Bot a question, uses “Can I Afford This?” to evaluate a planned purchase, or runs the “AI Health” check to receive an overall assessment of their financial condition based on their current data.

Scenario 6: Monthly Statement and Reporting

At the end of the month the user opens the Monthly Statement to review a summary of the period, and can download the project files as a ZIP package.

System Architecture :

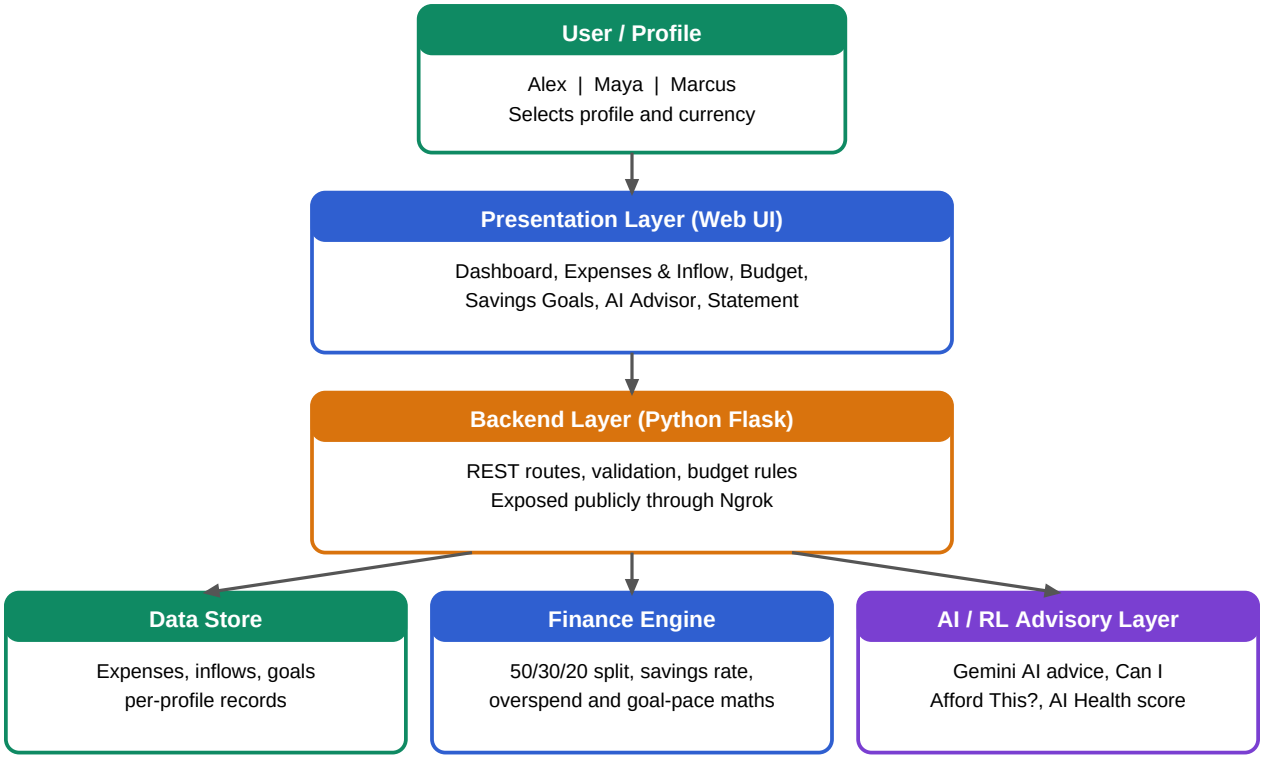


Figure: Layered architecture of the Personal Finance Advisor Bot

High-Level Flow

User → Profile Selection → Web Interface → Flask Backend → Finance Engine / AI Advisor → Result → Dashboard / Statement

Project Workflow

Milestone 1: Requirement Analysis and Architecture

- Activity 1.1: Identify the personal finance problem (untracked spending, lifestyle creep, weak savings habits) and define user requirements.
- Activity 1.2: Select the modules: dashboard, expense and inflow tracker, 50/30/20 budget, savings goals, AI advisor bot and monthly statement.
- Activity 1.3: Define the interaction between the web interface, Flask backend, AI service and data storage.
- Activity 1.4: Define the user profiles (Alex, Maya, Marcus) and currency options.

Milestone 2: Core Financial Features

- Activity 2.1: Implement expense and inflow logging with categories and payment methods.
- Activity 2.2: Implement 50/30/20 allocation with per-category monthly limits and spent-percentage indicators.
- Activity 2.3: Implement savings rate, net cash flow and overspending detection.
- Activity 2.4: Implement savings goals with progress and required monthly pace.

Milestone 3: AI and Reinforcement Learning Integration

- Activity 3.1: Integrate the AI Advisor Bot using Gemini AI for real-time financial advice.
- Activity 3.2: Implement “Can I Afford This?” and “AI Health” features.
- Activity 3.3: Frame advice as an RL loop: budget status as state, recommendation as action, improved savings behaviour as reward.

Milestone 4: Frontend Development

- Activity 4.1: Develop the responsive dashboard, tabs, cards, progress bars and forms.
- Activity 4.2: Apply the dark-themed interface with colour-coded status indicators (green, amber, red).

Milestone 5: Testing and Deployment

- Activity 5.1: Test input validation, budget calculations and the different user profiles.
- Activity 5.2: Test AI responses and error handling.
- Activity 5.3: Run the Flask application locally and expose it through Ngrok for demonstration.

Module 1 – Dashboard

The dashboard is the landing view of the platform. After a profile is selected, it greets the user with a short profile description and presents four key indicators together with any overspending alert. The Project Hub bar at the top also lets the user switch between related projects (Personal Finance Advisor Bot, Smart Traffic Signal RL and Hospital Emergency Management), and provides Project Report, Report PDF and Download ZIP options.

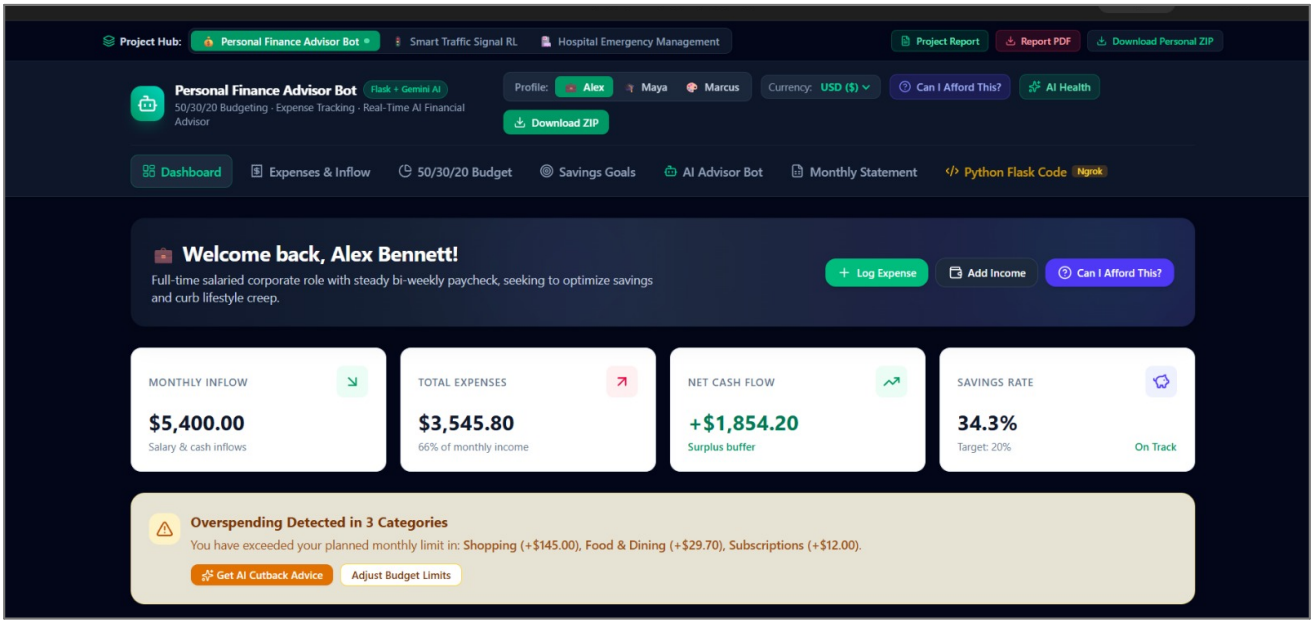


Figure 1: Dashboard for the Alex profile showing inflow, expenses, net cash flow, savings rate and overspending alert

Dashboard Metrics

Metric	Value shown (Alex)	Meaning
Monthly Inflow	$5,400.00	Salary and cash inflows for the month
Total Expenses	$3,545.80	66% of monthly income
Net Cash Flow	+$1,854.20	Surplus buffer after expenses
Savings Rate	34.3%	Above the 20% target (On Track)

Overspending Alert

The alert reports that three categories exceeded their limits: Shopping (+$145.00), Food & Dining (+$29.70) and Subscriptions (+$12.00). Two action buttons, *Get AI Cutback Advice* and *Adjust Budget Limits*, let the user respond immediately.

Quick Actions

- Log Expense
- Add Income
- Can I Afford This?

Module 2 – Expenses & Inflow

The Expenses & Inflow module is the transaction ledger. Each row records the date, a description, a category tag, the payment method and the amount, and it can be removed with the delete icon. Categories are colour-coded so that spending patterns are easy to scan. The screenshot below shows the Maya (student) profile.

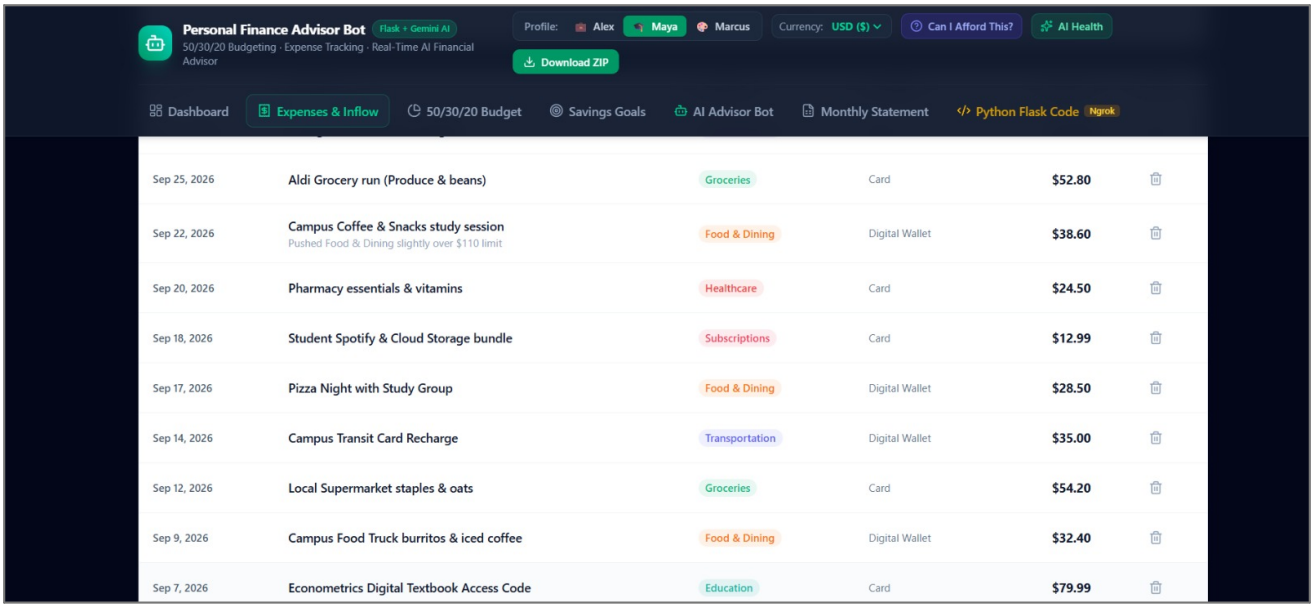


Figure 2: Expenses & Inflow list for the Maya profile

Input	Output
Date	Dated transaction list
Description	Category tag (colour-coded)
Category	Payment method label
Payment Method (Card / Digital Wallet)	Updated spending totals
Amount	Budget usage and overspending notes
Optional note	Dashboard and statement update

Sample Entries (Maya)

Date	Description	Category	Amount
Sep 25, 2026	Aldi Grocery run (Produce & beans)	Groceries	$52.80
Sep 22, 2026	Campus Coffee & Snacks study session	Food & Dining	$38.60
Sep 20, 2026	Pharmacy essentials & vitamins	Healthcare	$24.50
Sep 18, 2026	Student Spotify & Cloud Storage bundle	Subscriptions	$12.99
Sep 7, 2026	Econometrics Digital Textbook Access Code	Education	$79.99

Module 3 – 50/30/20 Budget

The 50/30/20 rule divides income into 50% Needs, 30% Wants and 20% Savings. The budget module applies this idea by tagging every category as a Need or a Want, showing spending to date, and comparing it with an editable monthly limit. Progress bars change colour as the limit approaches: green for comfortable usage, amber when usage is high, and full when a limit is reached. A *Match* button next to each limit field allows the limit to be aligned quickly.

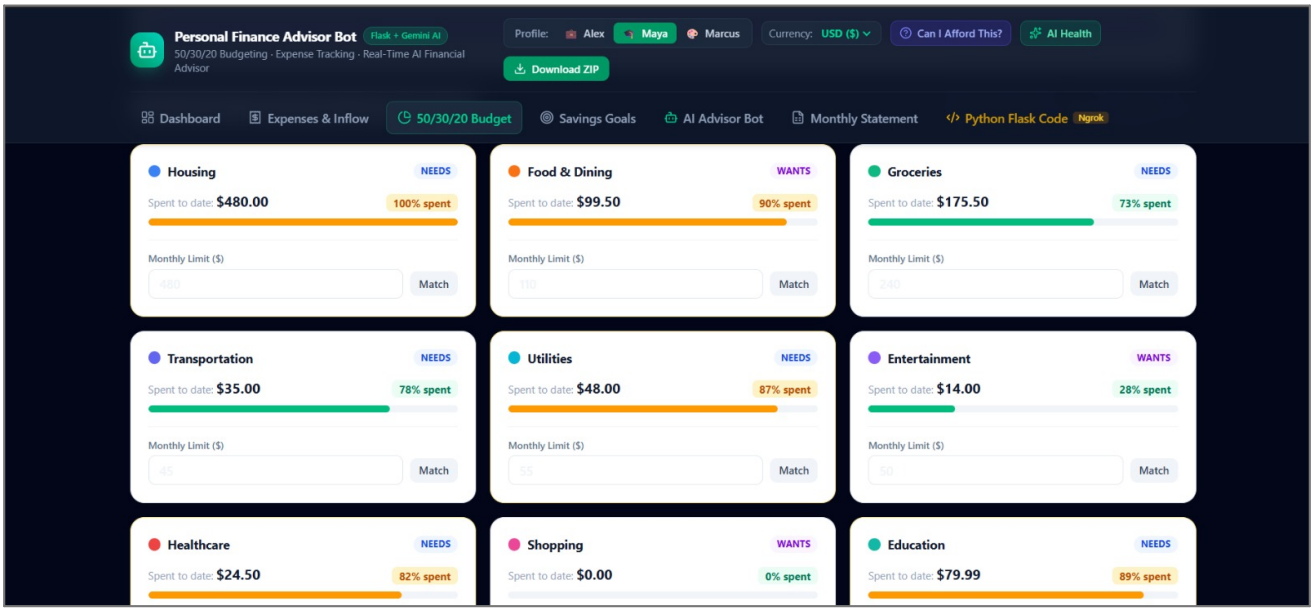


Figure 3: 50/30/20 budget cards for the Maya profile

Category	Type	Spent to date	Monthly limit	Used
Housing	Needs	$480.00	$480	100%
Food & Dining	Wants	$99.50	$110	90%
Groceries	Needs	$175.50	$240	73%
Transportation	Needs	$35.00	$45	78%
Utilities	Needs	$48.00	$55	87%
Entertainment	Wants	$14.00	$50	28%
Healthcare	Needs	$24.50	–	82%
Shopping	Wants	$0.00	–	0%
Education	Needs	$79.99	–	89%

Healthcare, Shopping and Education limits are not fully visible in the screenshot, so they are shown as –.

Module 4 – Savings Goals

The Goal-Based Savings Hub helps users track emergency buffers, debt payoffs and purchase funds. An overall portfolio card shows the total saved across all active goals, and each goal card shows the amount saved, target, percentage achieved, target date and the monthly pace required to finish on time.

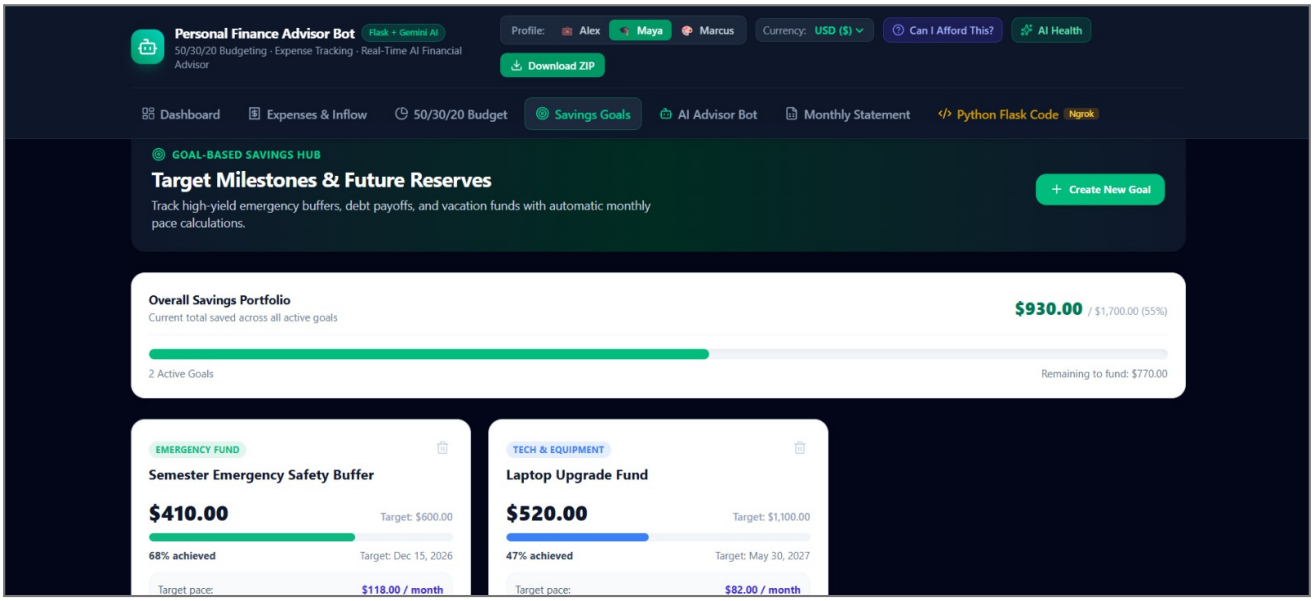


Figure 4: Savings Goals hub for the Maya profile

Goal	Type	Saved	Target	Achieved	Target date	Monthly pace
Semester Emergency Safety Buffer	Emergency Fund	$410.00	$600.00	68%	Dec 15, 2026	$118.00
Laptop Upgrade Fund	Tech & Equipment	$520.00	$1,100.00	47%	May 30, 2027	$82.00

Portfolio Summary

- Total saved across 2 active goals: $930.00 of $1,700.00 (55%)
- Remaining to fund: $770.00
- Create New Goal button for adding more targets

Monthly Pace Logic

Required monthly pace = (Target amount - Amount saved) ÷ Months remaining until the target date.

Module 5 – AI Advisor Bot, Can I Afford This? and AI Health

The AI layer turns raw numbers into advice. The AI Advisor Bot provides a conversational interface for real-time financial guidance using Gemini AI through the Flask backend. The header also provides two shortcut features that are available from every tab.

Features

- **AI Advisor Bot** – ask questions about budgeting, spending and saving.
- **Can I Afford This?** – evaluate a planned purchase against current budget and savings.
- **AI Health** – overall assessment of the user's financial condition.
- **Get AI Cutback Advice** – suggestions for reducing spending in overspent categories.

Reinforcement Learning Perspective

RL element	In this platform
State	Current budget usage, savings rate, goals and profile
Action	A recommendation such as reduce a category, move money to a goal or delay a purchase
Reward	Improved savings rate, fewer overspent categories, goals staying on pace
Policy	The advisor's strategy for choosing the best recommendation for each state

The Project Hub also lists Smart Traffic Signal RL, showing that the same platform is used for other RL-based projects. The RL elements above describe the advisory design of this project; the visible implementation uses Gemini AI for generating advice.

Fallback Mechanism

If the AI service cannot be reached, the application should still allow the user to continue tracking expenses, budgets and goals, because these features are calculated by the application itself and do not depend on the AI response.

Module 6 – Monthly Statement

The Monthly Statement tab collects the month's inflows, expenses, category totals and savings position into a single reviewable summary. The Report PDF and Download ZIP buttons allow the results and project files to be exported.

Data Flow and API Integration

Data Handling

The user's selected profile and currency determine which records are loaded. Transactions, budget limits and goals are stored per profile so that Alex, Maya and Marcus each see their own data.

Python Flask Code

The Python Flask Code tab displays the backend source code and is tagged with Ngrok, which is used to make the locally running Flask server reachable through a public URL for demonstration.

AI Request Endpoint

Questions from the AI Advisor Bot, "Can I Afford This?" and "AI Health" are sent from the frontend to the Flask backend, which forwards the request with the user's financial context to Gemini AI and returns the response to the interface.

Integration Flow

```
Log Expense / Add Income
      ↓
Input Validation
      ↓
Flask Backend + Data Store
      ↓
Budget / Savings / Cash-Flow Calculation
      ↓
Dashboard + Budget + Goals Update

AI Advisor / Can I Afford This? / AI Health
      ↓
Flask API route
      ↓
Gemini AI
      ↓
Advice Response
      ↓
Chat / Result Interface
```

Project Structure

An illustrative structure for a Flask-based project of this type is shown below.

```
personal-finance-advisor-bot/
├── app.py                 # Flask backend
├── requirements.txt
├── templates/
│   └── index.html
├── static/
│   ├── style.css
│   └── script.js
├── data/                  # profile data
└── README.md
```

- **app.py** – Flask application that handles routes, calculations, storage and AI requests.
- **requirements.txt** – Python packages required, such as Flask and the Gemini AI client.
- **index.html** – structure of the interface: tabs, forms, cards and tables.

- **style.css** – dark theme, cards, progress bars, badges and responsive layout.
- **script.js** – tab navigation, profile and currency switching, form handling and updates.
- **README.md** – overview, setup steps and usage instructions.

User Interface and Design

The interface uses a modern dark theme with white content cards, rounded corners and clear colour accents. A top bar provides profile selection, currency selection and the main shortcuts, and a tab bar provides access to the seven main views.

Design Principles

- Simple tab-based navigation between finance tools.
- Card-based presentation of totals, budgets and goals.
- Colour-coded categories and status labels (Needs, Wants, spent percentage).
- Progress bars for budget usage and savings goals.
- Profile switching for different financial situations.
- Clear call-to-action buttons such as Log Expense and Add Income.

Colour Meaning

Colour	Meaning
Green	Comfortable usage or positive result (for example savings rate)
Amber / Orange	High usage close to the limit, or warning
Red	Overspending or high-priority alert
Blue / Purple	Informational or AI-related actions

Testing Strategy

Functional Testing

- Verify that expenses and inflows are added and deleted correctly.
- Verify net cash flow and savings rate calculations.
- Verify spent percentages against monthly limits.
- Verify overspending alerts when limits are exceeded.
- Verify goal progress and monthly pace calculation.
- Verify that each profile shows its own data.
- Verify AI responses and failure handling.

Boundary Testing

- Zero spending in a category (0%).
- Spending exactly at the limit (100%).
- Spending above the limit.
- Goal with target date close to or in the past.
- Empty or non-numeric amounts.

Non-Functional Testing

- Responsiveness
- Usability
- Error handling
- Security of API keys
- Performance of calculations

Deployment

The application is developed in Python using Flask. For demonstration it is run locally and exposed through Ngrok, which creates a public URL to the local server. This allows the platform to be opened from any browser during demonstration without full cloud hosting.

Execution

After the Flask server is started, the Ngrok public link can be opened in a browser. Users can then select a profile, log expenses and income, review the 50/30/20 budget, track savings goals, use the AI Advisor Bot and open the monthly statement.

The platform was tested by switching between the Alex and Maya profiles and navigating through the Dashboard, Expenses & Inflow, 50/30/20 Budget and Savings Goals tabs, as shown in the screenshots in this document.

Conclusion

The Personal Finance Advisor Bot combines expense tracking, 50/30/20 budgeting, goal-based savings, AI-generated advice and dashboard visualisation in one responsive web interface. Its modular design allows users to move from logging a transaction to understanding its effect on their budget and goals without switching between separate tools.

The project demonstrates how web development, financial rules, AI assistance and a reinforcement-learning perspective on recommendations can be combined into a practical personal finance platform. Future work can include training an RL policy on real user feedback, adding bank-statement import, and adding stronger security and authentication before production use.